

1. Algoritmo Metropolis-Hastings

Denotemos $x = (x_1, x_2)^T \in \mathbb{R}^2$. Haz una implementación del algoritmo Metrópolis-Hastings para tomar muestras de una distribución objetivo

$$\pi(x) \propto \exp \left(-10(x_1^2 - x_2)^2 - \left(x_2 - \frac{1}{4}\right)^4 \right) \quad (1)$$

usando como propuesta

$$q(x, x') \propto \exp \left(-\frac{1}{2\gamma^2} \|x - x'\|^2 \right) \quad (2)$$

El algoritmo Metropolis-Hastings se describe a continuación:

Algoritmo 1: Metropolis-Hastings

Input: Punto inicial $x^{(0)}$, número de iteraciones N

Output: $\{x^{(1)}, x^{(2)}, \dots, x^{(N)}\}$

for $k = 0, 1, 2, \dots, N - 1$ **do**

 Proponer $x' \sim q(x^{(k)}, \cdot)$;

 Calcular el logaritmo del cociente de aceptación:

$$\log \alpha(x^{(k)}, x') = \min \left(0, \log \pi(x') - \log \pi(x^{(k)}) + \log q(x', x^{(k)}) - \log q(x^{(k)}, x') \right)$$

 Muestrear $u \sim U(0, 1)$;

if $\log u \leq \log \alpha(x^{(k)}, x')$ **then**

 | Aceptar: $x^{(k+1)} \leftarrow x'$;

else

 | Rechazar: $x^{(k+1)} \leftarrow x^{(k)}$;

end

end

return $\{x^{(1)}, x^{(2)}, \dots, x^{(N)}\}$

siguiendo las siguientes instrucciones:

- Transforma el cálculo del cociente de aceptación $\alpha(x, x')$ a escala logarítmica. Esto evita la inestabilidad numérica ocasionada por calcular el cociente de dos números en $[0, 1]$. Nota que no es necesario conocer la constante de normalización de $\pi(x)$.
- Genera 10000 muestras de la posterior usando $\gamma = 0.001$, $\gamma = 0.05$, $\gamma = 0.1$ y $\gamma = 1$. Descarta los primeros 5000 pasos de la caminata.
- En cada caso, haz una gráfica de $-\log(\pi(x))$, de la caminata y las curvas de nivel de $\pi(x)$ (no importa que no conozcas la constante de normalización).

- (d) Usa la librería **corner** de Python para representar el histograma obtenido con el muestreo.
- (e) Tiempo integrado de autocorrelación. Dada una distribución $\pi(x)$, podemos calcular la esperanza de una función con respecto de la medida asociada a la distribución

$$\mathbb{E}_{\pi(x)}[f(x)] = \int_{\mathbb{R}^2} f(x)\pi(x) dx$$

Si $\{x^{(1)}, x^{(2)}, \dots, x^{(N)}\}$ son muestras independientes de la distribución objetivo, entonces podemos aproximar la esperanza mediante el promedio ergódico

$$\mathbb{E}_{\pi(x)}[f(x)] = \frac{1}{N} \sum_{i=1}^N f(x^{(i)}) + o\left(\frac{1}{\sqrt{N}}\right),$$

y similarmente para la varianza

$$\sigma^2 = \frac{1}{N} \text{Var}_{\pi(x)}[f(x)].$$

Cuando las muestras $\{x^{(1)}, x^{(2)}, \dots, x^{(N)}\}$ se obtienen mediante Markov Chain Monte Carlo no son independientes y se cumple

$$\sigma^2 = \frac{\tau_f}{N} \text{Var}_{\pi(x)}[f(x)],$$

donde τ_f se define como el tiempo integrado de autocorrelación, el cuál se interpreta como el número de pasos necesarios para obtener la siguiente muestra independiente.

Usa el programa `pytwalk.py` para tomar muestras de la distribución (1) y calcula el tiempo integrado de correlación usando el método `.Ana()`.

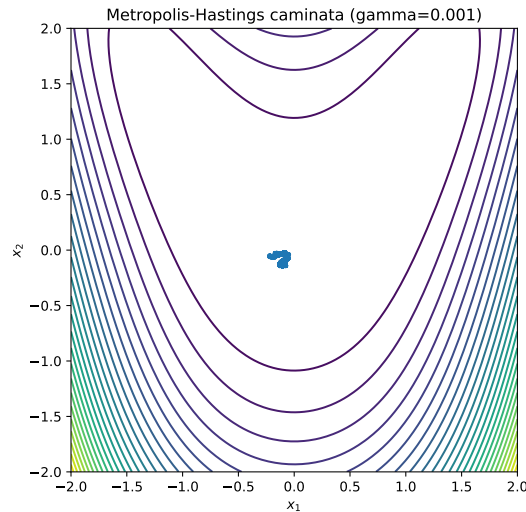


Figura 0.1: inciso (c) ($\gamma = 0.001$)

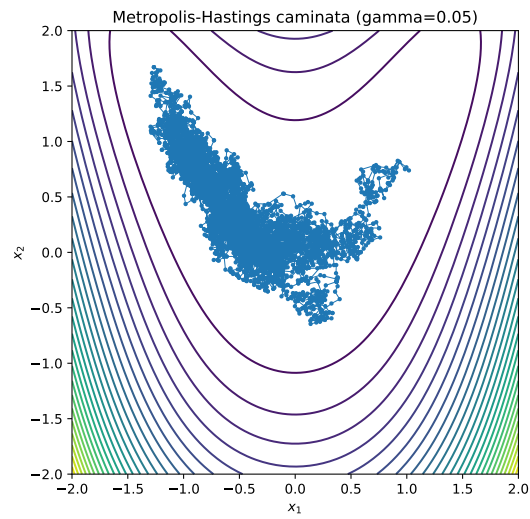


Figura 0.2: inciso (c) ($\gamma = 0.05$)

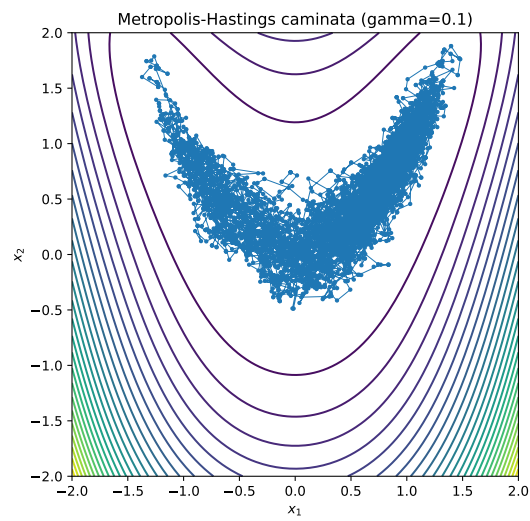


Figura 0.3: inciso (c) ($\gamma = 0.1$)

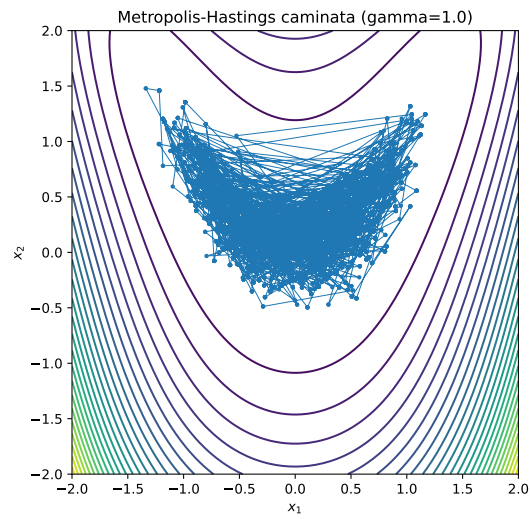


Figura 0.4: inciso (c) ($\gamma = 1$)

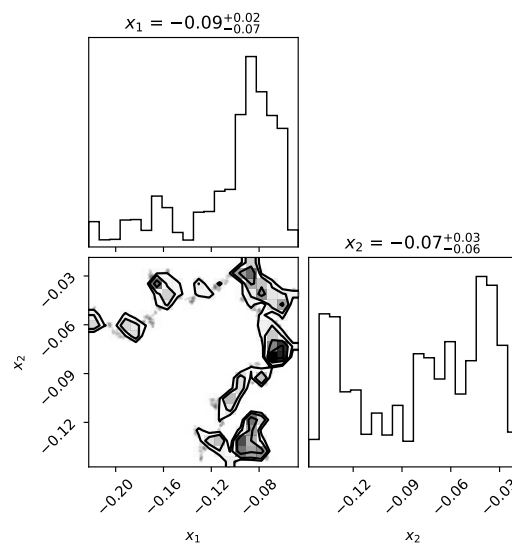


Figura 0.5: inciso (d) ($\gamma = 0.001$)

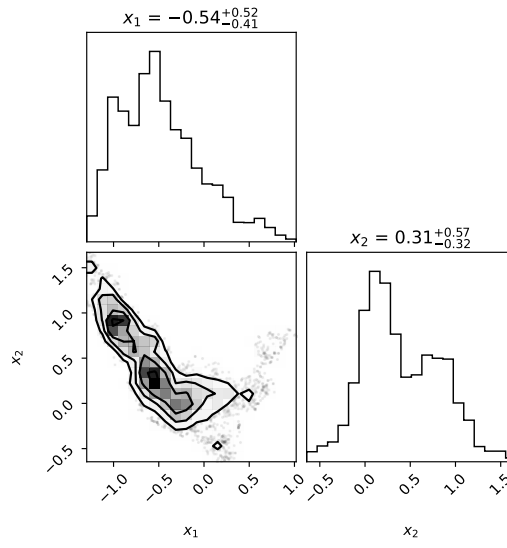


Figura 0.6: inciso (d) ($\gamma = 0.05$)

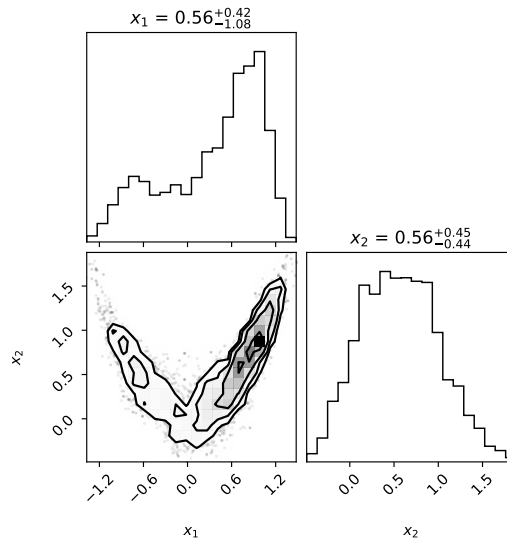
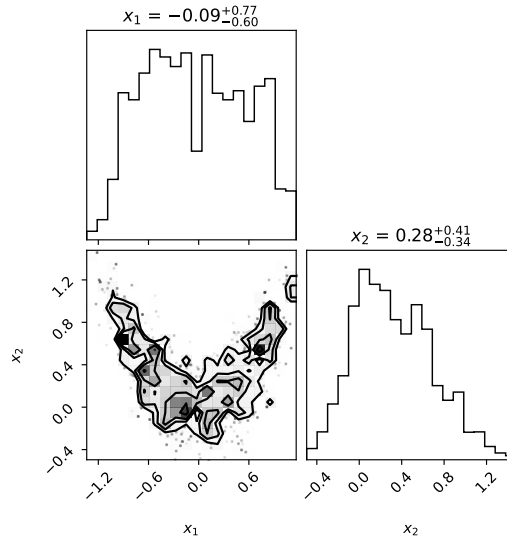


Figura 0.7: inciso (d) ($\gamma = 0.1$)


 Figura 0.8: inciso (d) ($\gamma = 1$)

Reporte de (b), (c) y (d)

Se realizaron simulaciones del algoritmo de Metropolis-Hastings para distintos valores del parámetro γ , el cual controla la varianza de la propuesta gaussiana.

Cuando γ toma valores muy pequeños (por ejemplo $\gamma = 0.001$), las propuestas generan desplazamientos muy pequeños alrededor del estado actual. Como consecuencia, la cadena de Markov se mueve lentamente y explora únicamente una pequeña región del espacio de estados. Esto produce una caminata muy concentrada que no representa adecuadamente la distribución objetivo $\pi(x)$.

Para valores intermedios de γ (por ejemplo $\gamma = 0.05$ y $\gamma = 0.1$), la caminata comienza a explorar mejor el espacio. En estos casos las muestras generadas siguen de forma más clara las regiones de mayor probabilidad de la distribución $\pi(x)$, lo cual también puede observarse en las curvas de nivel y en los histogramas obtenidos con `corner`. En este rango la cadena presenta un equilibrio entre exploración y tasa de aceptación.

Cuando γ toma valores grandes (por ejemplo $\gamma = 1$), las propuestas generan saltos muy grandes en el espacio de estados. Esto provoca que muchas propuestas caigan en regiones de baja probabilidad y por lo tanto sean rechazadas con mayor frecuencia. Como resultado, la cadena puede volverse más dispersa y con menor eficiencia en la exploración de la distribución objetivo.

En conclusión, el parámetro γ controla el tamaño de los pasos de la caminata de Markov. Valores muy pequeños producen una exploración lenta del espacio, mientras que valores muy grandes generan muchas propuestas rechazadas. Un valor intermedio permite que la cadena reproduzca de forma más adecuada la estructura de la distribución $\pi(x)$.

(e) Tiempo integrado de autocorrelación

Se utilizó la librería `pytwalk` para generar muestras de la distribución objetivo y estimar el tiempo integrado de autocorrelación mediante el método `.Ana()`. Los resultados obtenidos fueron:

- Tasas de aceptación para los kernels Walk, Traverse, Blow y Hop:

$$(0.3597, 0.1584, 0.1818, 0.0000)$$

- Tasa de aceptación global:

$$0.2591$$

- Máximo rezago usado para estimar la autocorrelación:

$$\text{maxlag} = 110$$

- Tiempo integrado de autocorrelación:

$$\tau = 40.3$$

- Tiempo integrado de autocorrelación normalizado:

$$\frac{\tau}{n} = 20.1$$

El tiempo integrado de autocorrelación τ indica que aproximadamente cada 40 iteraciones de la cadena se obtiene una muestra aproximadamente independiente de la distribución objetivo.